Week 8 Technical Report - Computational Complexity

Project repository (complete source, tests, benchmarks and results):
https://github.com/RDeibel2025/CTX-Advanced-Algorithms
Computational complexity: src/complexity/ · demonstration: examples/week8_demo.py
Benchmark: benchmarks/week8_complexity_benchmark.py

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026

📎 Complete project repository

https://github.com/RDeibel2025/CTX-Advanced-Algorithms

1. Executive Summary

I built a reduction framework that maps 3-SAT to vertex cover and on to independent set and clique, exact solvers for subset sum, SAT and vertex cover, a vertex cover 2-approximation, nearest-neighbor TSP with 2-opt, and greedy and local-search heuristics, and benchmarked them on seeded instances. The report reads every result through one fact: checking a solution is cheap; finding one is not. Worst-case subset-sum backtracking visited 20,530,286 nodes at n = 28, growing by a factor of 1.84 per added value, while checking a subset-sum certificate took 9.6 to 47.1 microseconds at every size from 10 to 40. The 2-approximation stayed inside its proven bound, averaging 1.575 and reaching exactly 2 on K3,3 and the star, while nearest neighbor has no bound and cost 62 against an optimum of 48 on a six-city line.

2. Methodology

Machine and timing. Apple M2 Max, 64 GB RAM, macOS 14.5 (arm64), CPython 3.12.4. Each call is timed with `time.perf_counter` with garbage collection paused; a call under 20 ms is retimed as the mean of 5 runs. Each verifier time is the mean of 20 runs of the polynomial-time checker on the certificate the method returned: `is_satisfying`, `is_vertex_cover`, a sub-multiset and sum check for subset sum, and a permutation and tour-cost check for TSP.

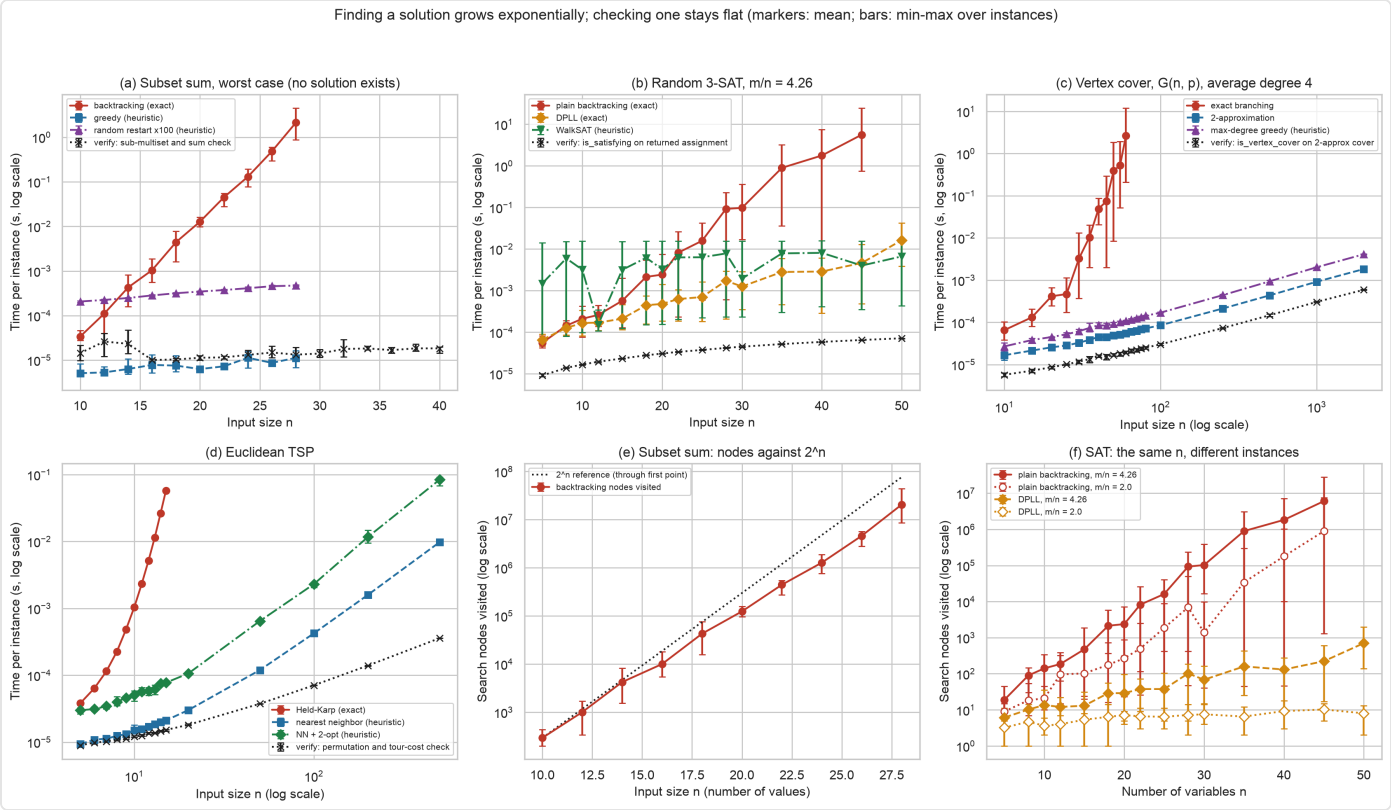
Instances. Subset sum: 5 instances per n = 10, 12, ..., 40, in two families. The worst case uses even values up to 2,000,000 and an odd target, so no subset exists and backtracking cannot stop early. The satisfiable family sets the target to the sum of a random subset. SAT: 10 random 3-SAT formulas per n = 5 to 50, at clause ratios 4.26 and 2.0. Vertex cover: 5 G(n, p) graphs per n with average degree 4, n = 10 to 2,000, plus the path, star and K3,3. TSP: Euclidean points in a 100 x 100 square, 5 instances per n = 5 to 15 with the Held-Karp optimum from Week 6, and 3 per n = 20 to 500. Reduction: 3 satisfiable formulas per n = 10 to 1,000 at ratio 2.0. WalkSAT gets 10,000 flips with noise 0.5, and random-restart greedy 100 tries.

Seeds. The master seed is 42 (`--seed`). Every instance comes from `derive_seed(42, experiment, ..., n, rep)`, a blake2b digest, and the instance CSV records 916 distinct seeds.

The 30-second rule. An exact solver's sweep stops before the next size when a run at the current size passed 30 s, or when the growth measured so far predicts it would. Four sweeps stopped: subset-sum backtracking after n = 28 (34.4 s predicted at n = 30), plain SAT backtracking after n = 45 at both ratios (82.1 s and 39.2 s predicted at n = 50), and exact vertex cover after n = 60 (75.4 s predicted at n = 65). Every stop is recorded in `sweep_limits.csv`, and no point was projected: all 275 rows are measured.

Metrics and outputs. Approximation ratio is the method's size or cost divided by the exact optimum. Success rates carry 95% Wilson intervals. The repository holds every week's project, and the root-level `comparison_table.csv` is Week 6's, so Week 8's outputs live in `benchmarks/results/week8/` under the names the instructions specify.

3. Results



3.1 Verification against search

On the worst-case family, backtracking's mean time rose from 34 microseconds at n = 10 to 2.11 s at n = 28 (largest single run 4.53 s), and nodes from 292 to 20,530,286. Fitted over all sizes, nodes grew by 1.836 per added value, against 2.0 for a full binary search tree. Greedy took 5.1 to 11.3 microseconds and random restart 207 to 471 microseconds at every size. A NO answer here has no short certificate to check. On the satisfiable family, backtracking found all 80 subsets, and the sum check took 9.6 to 47.1 microseconds.

The other problems show the same gap. At ratio 4.26, plain SAT backtracking reached 5.60 s and 6,036,672 nodes at n = 45, growing by 1.378 per variable, while `is_satisfying` took 8.5 to 72.6 microseconds. Exact vertex cover reached 2.67 s at n = 60, growing by 1.253 per vertex, against 19.8 microseconds to verify its

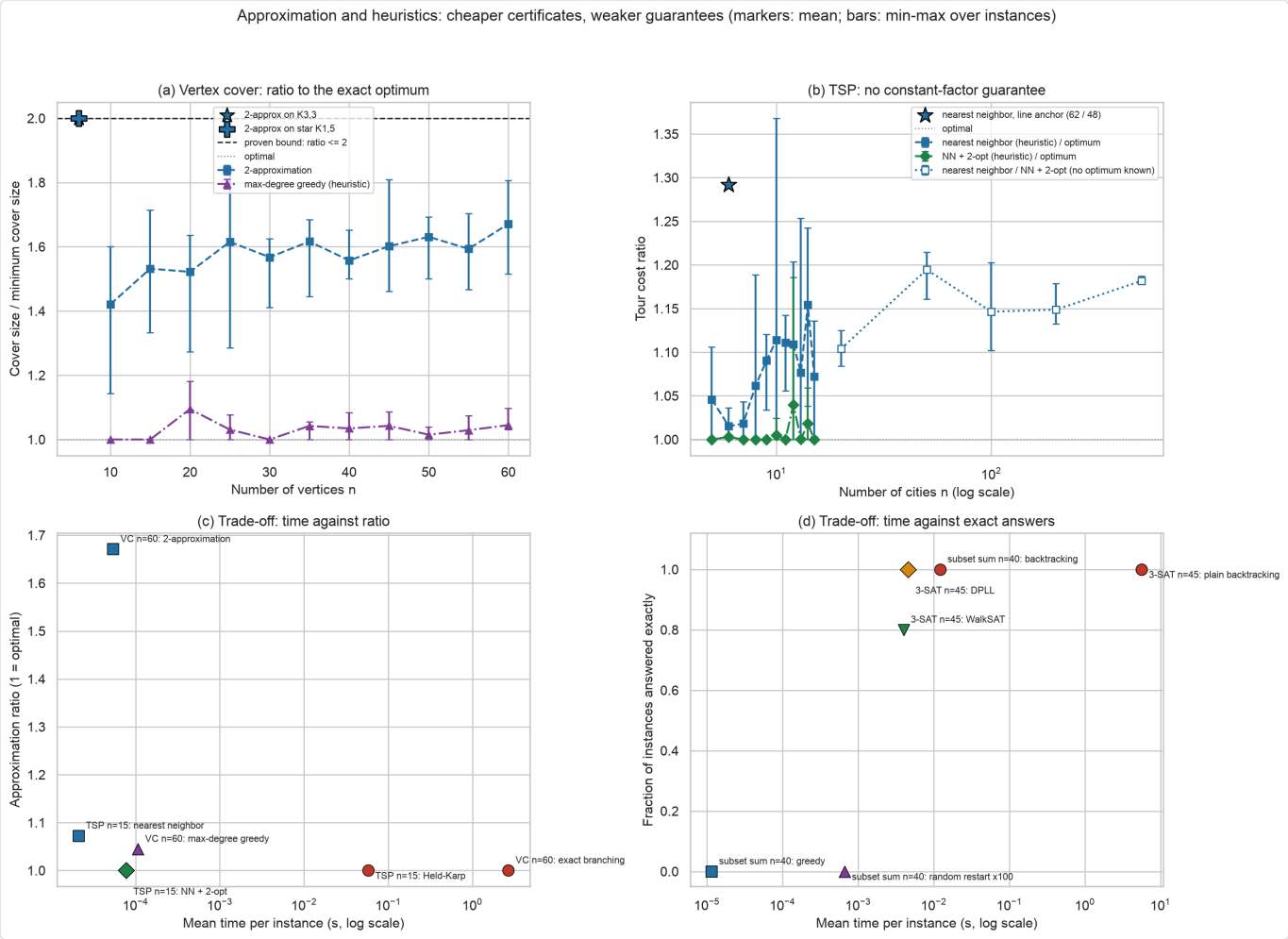
cover, a ratio of 134,646x. Held-Karp grew by 2.224 per city to 0.0576 s at n = 15, against 14.9 microseconds for the tour check. Across 1,564 certificates, every verifier took 3.0 to 2,367.5 microseconds.

The curve also depends on the instance (panel f): at n = 45, plain backtracking visited 6,036,672 nodes at ratio 4.26 and 891,181 at ratio 2.0, and DPLL 222 and 10.

3.2 The reduction

The 3-SAT to vertex cover construction (Garey & Johnson, 1979) produced 2n + 3m nodes and n + 6m edges every time: 8,000 nodes and 13,000 edges with k = 5,000 at n = 1,000, in 13.3 ms. Its cost stayed between 1.6 and 1.9 microseconds per node from n = 10 to 1,000, which is linear. In all 15 runs, a WalkSAT assignment mapped forward to a vertex cover of size exactly k, passed `is_vertex_cover`, and mapped back to a satisfying assignment. Tests confirm the anchors: 13 nodes, 20 edges and k = 8 for the three-clause formula, and a minimum cover of 20 against k = 19 for the unsatisfiable eight-clause one.

3.3 Approximation



Over 55 graphs with a known optimum, the 2-approximation's ratio averaged 1.575 (1.143 to 1.846) and was never optimal. It was exactly 2.0 on the path, the star and K3,3. Max-degree greedy, with no guarantee, averaged 1.030 (at most 1.182) and was optimal on 28 graphs. The 2-approximation also certifies a lower bound: its matching has 835 edges at n = 2,000, so every cover has at least 835 vertices, and greedy's 1,100.6

is provably within 1.318 of optimal. Nearest neighbor averaged 1.079 over 55 TSP instances (at most 1.368) and was optimal on 11. 2-opt brought the mean to 1.006 and made 47 optimal. At n = 500, 2-opt took 83.7 ms against nearest neighbor's 9.74 ms and nearest neighbor's tours were 1.182 times as long as the 2-opt tours.

3.4 Heuristics

WalkSAT (Selman et al., 1994) solved all 105 satisfiable formulas at ratio 4.26 (95% interval 0.965 to 1.000) but returned nothing on the 45 unsatisfiable ones, which it cannot prove unsatisfiable. On satisfiable subset-sum instances, greedy hit the target on 2 of 80 (0.025, interval 0.007 to 0.087) with a mean gap of 25,903, and random restart on 7 of 80 (0.087) with a mean gap of 800.6.

4. Discussion

NP is the class of problems whose YES answers have certificates a polynomial-time verifier can check, (CLRS Ch. 34; Amakobe, 2025, Ch. 8), and the measurements show that side of the definition directly. No polynomial algorithm is known for any NP-complete problem. Reductions explain why finding one would settle all of them. The reduction built here ran in linear time and carried certificates both ways. So a polynomial vertex cover solver, run after the 13.3 ms transformation, would decide 3-SAT in polynomial time, and Cook (1971) and Karp (1972) show that every NP problem reduces to 3-SAT. P = NP would follow from one fast algorithm for any of them.

The exponential curves explode because each added item doubles the space of candidate solutions. Pruning changes the base, not the shape: 1.84 per value for subset sum, 1.38 per variable for plain SAT backtracking, 1.10 for DPLL at ratio 4.26, and 1.25 per vertex for exact vertex cover. Any base above 1 is still exponential. Subset sum is weakly NP-complete. Its dynamic program runs in O(n · target), polynomial in the target's value but exponential in the target's length in bits, and the values here run up to 2,000,000.

Approximation and heuristics give up optimality to get a certificate quickly. The difference between them is the guarantee. The 2-approximation's factor follows from the matching argument (CLRS §35.1): the picked edges share no endpoints, any cover needs one endpoint of each, so OPT ≥ |M| and the algorithm returns 2|M|. Greedy did better on these random graphs, but its worst cases are built families that this benchmark did not generate, and only the 2-approximation can certify its own quality. Nearest neighbor's line instance shows the same thing for TSP.

5. Conclusion

Knowing that a problem is NP-complete changes the design question from how to solve it exactly to which guarantee can be afforded at the needed size. At n = 60, the exact vertex cover took 2.67 s per graph, and the next size was predicted at 75.4 s. The 2-approximation took 54 microseconds and returned a cover no more than twice the minimum.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Ch. 34 NP-Completeness; Ch. 35 Approximation Algorithms, §35.1 The vertex-cover problem.
- Amakobe, M. (2025). *Advanced Computational Algorithms*, Ch. 8 (open textbook).
- Cook, S. A. (1971). The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151-158.
- Karp, R. M. (1972). Reducibility among combinatorial problems. In R. E. Miller & J. W. Thatcher (Eds.), *Complexity of Computer Computations* (pp. 85-103). Plenum.
- Garey, M. R., & Johnson, D. S. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- Selman, B., Kautz, H., & Cohen, B. (1994). Noise strategies for improving local search. *AAAI-94*, 337-343.

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading, including the hand-checked values the tests assert. Every measurement comes from running the benchmark on my machine, and every number in this report is printed from the CSVs by `tools/week8_facts.py`. Full disclosure: `docs/AI_USE.md`.